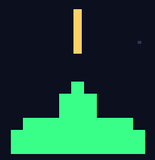


REINFORCEMENT LEARNING · PROJECT PROPOSAL

Rainbow DQN on Space Invaders

A value-based agent in our own re-implementation of the arcade classic

Sergey Peshkov · Anton Kropotov · Arina Nikolaeva
Innopolis University · Reinforcement Learning, Fall 2026



5–7 MINUTES

Outline



1	**Algorithm overview** What Rainbow DQN is, its main idea, the six components and the update step	1–2 min
2	**Environment** Observations, actions, reward, goal, episode end, what the agent must learn	2–3 min
3	**Motivation & plan** Why this environment, why Rainbow fits it, challenges, how we train and evaluate	1–2 min

Rainbow DQN at a Glance

VALUE-BASED | OFF-POLICY | MODEL-FREE | DISCRETE ACTIONS

Base: DQN (Mnih et al., 2015)

- A CNN learns the action-value Q(s, a) from pixels
- Act greedily: a = argmax Q(s, a)
- Experience replay: learn from random past transitions
- Target network: a slow copy that makes the TD target stable

Rainbow (Hessel et al., 2018)

- Combines six independent DQN improvements in one agent
- Learns the whole distribution of returns Z(s, a), not only its mean
- Policy: argmax of the expected value E[Z(s, a)]
- Strongest DQN-family agent on the Atari benchmark

Off-policy: learns from replayed data collected by older policies. Model-free: never learns the game dynamics.

Six Components

Double Q-learning

Online net picks a*, target net evaluates it: removes overestimation of Q.

Prioritized replay

Sample transitions with large TD error more often (alpha = 0.5); correct bias with importance weights.

Dueling network

Q(s,a) = V(s) + A(s,a) - mean A: learns state value even when the action does not matter.

Multi-step returns

n = 3 rewards before bootstrapping: reward propagates faster.

Distributional (C51)

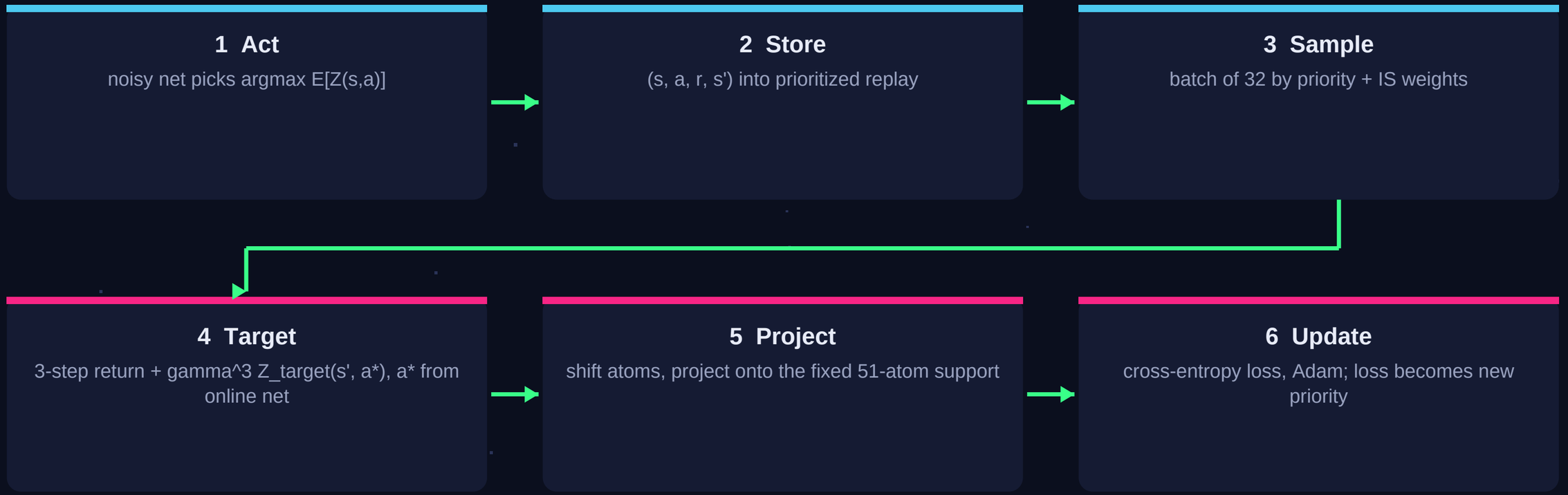
51 atoms on [-10, 10]: predicts a probability for each possible return.

Noisy Nets

Learned noise in the linear layers replaces epsilon-greedy exploration.

Paper ablation: removing prioritized replay or multi-step hurts most, then distributional; noisy, dueling and double add smaller gains.

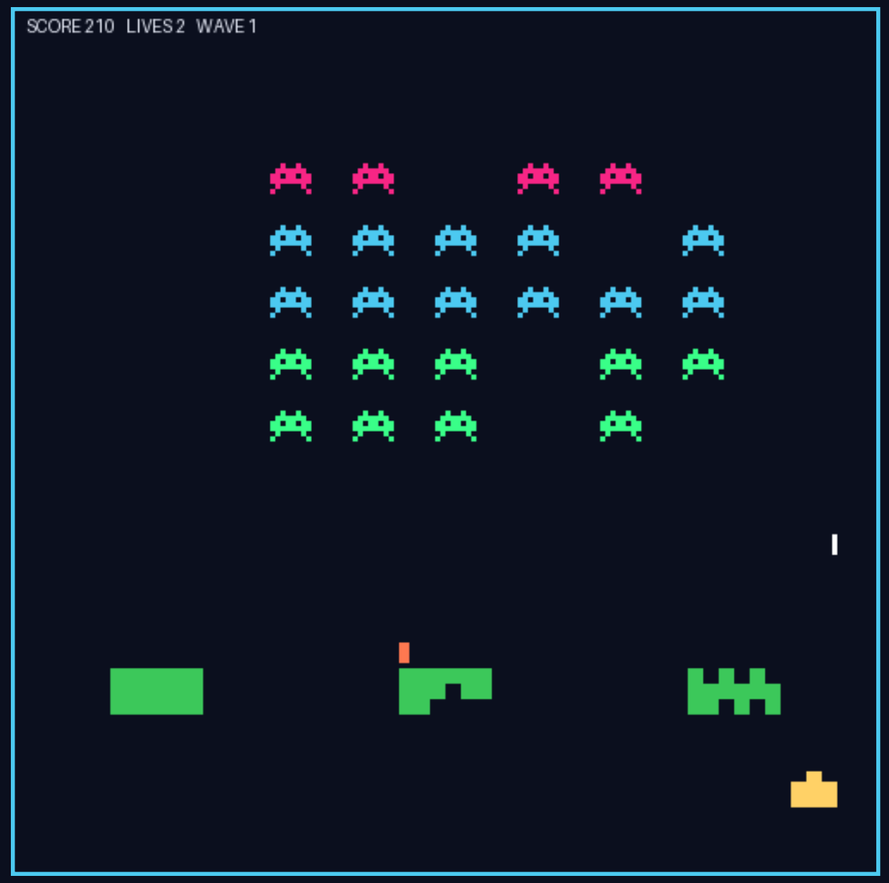
How One Learning Step Works



Every 4 environment steps: one update. Every 2,000 updates of the online net: copy it to the target net.

Loss: L = - sum_i m_i log p_theta(s, a)_i   (m = projected target distribution)

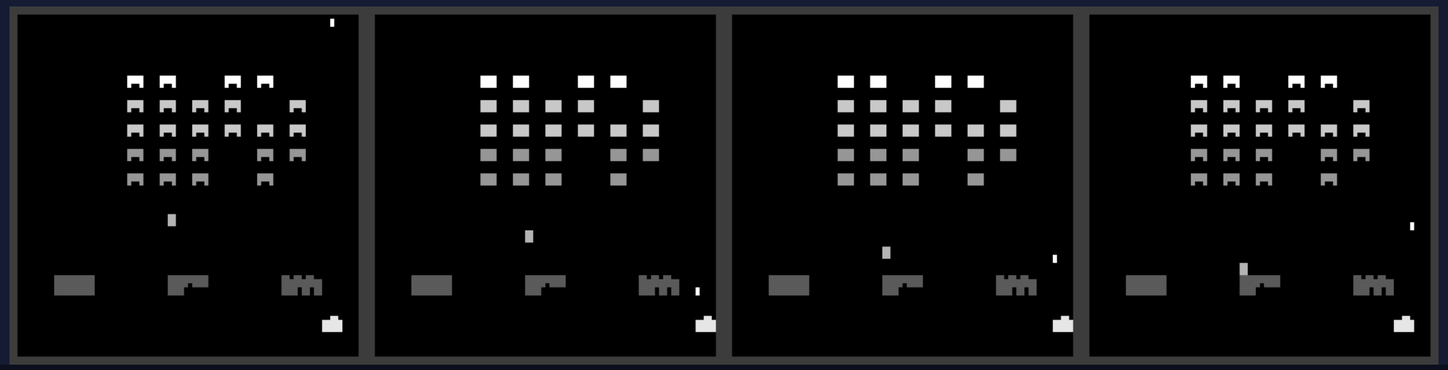
Environment: Space Invaders, Our Version



- **Same rules as the Atari game** — rows of invaders move and speed up, shoot back, shields erode, a UFO gives bonus points.
- **Written from scratch** — Python + NumPy, standard Gymnasium API (reset / step), about 1,200 steps/s headless.
- **Instrumented** — every game event (kill, hit, miss, dodge, wave cleared) is reported, so we can build reward triggers.
- **Configurable** — 3 difficulty levels: number of invaders, speed and fire rate.
- **Stochastic** — random enemy fire, UFO and 0–30 no-op frames at the start.

Observation and Action Space

Observation: 4 stacked grayscale frames, 4 x 84 x 84, values 0–255



One frame shows positions; the stack of 4 shows speed and direction of invaders and bullets (makes the state ~Markov).

Action space: Discrete(6), same as Atari

0	1	2	3	4	5
NOOP	FIRE	RIGHT	LEFT	RIGHT+FIRE	LEFT+FIRE

Reward, Goal and Episode End

Reward

Invader, bottom rows	10 pts
Invader, middle rows	20 pts
Invader, top row	30 pts
UFO	100 pts

Reward = game score of the 4 frames, clipped to [-1, 1] for training (as in DQN on Atari). Optional shaped variant: penalty for being hit or missing, bonus for dodging and clearing a wave.

Goal

- Maximise the game score: destroy as many invaders as possible without losing all 3 lives

Episode ends when

- all 3 lives are lost, or
- the invaders reach the player row, or
- time limit: 10,000 steps (truncation)

Discount gamma = 0.99. One agent step = 4 game frames (frame skip). After clearing a wave, a faster wave starts.

What the Agent Should Learn

From pixels only — nobody tells the agent where the invaders or bullets are.

Aim

Shoot when an invader is above, not randomly: only one player bullet can be on screen.

Dodge

Track enemy bullets in the frame stack and move away; use shields as cover.

Prioritise

Kill the lowest invaders first (they land soonest) and catch the UFO for 100 points.

Plan ahead

Rewards come later than the shot (bullet flight time), so the agent must assign credit over several steps.

Why This Environment?

- **Classic benchmark** — Space Invaders is one of the Atari games on which DQN and Rainbow were evaluated (paper: DQN 1,692 vs Rainbow 18,789 points).
- **Right difficulty** — pixel input, moving targets, stochastic enemy fire and delayed reward — hard, but trainable on one GPU in hours.
- **Full control** — our own implementation lets us read every game event, change rewards and difficulty, and train fast without an emulator.
- **Easy to judge** — the score is intuitive, and a video of the agent shows clearly what it has learned.

Why Rainbow Fits — and What Is Hard

Why Rainbow suits it

- Discrete 6-action space: exactly what Q-learning needs
- CNN on stacked frames, as in the original Atari work
- Replay + priorities reuse rare events (UFO, wave clear)
- Distributional head models the random enemy fire
- n-step returns bridge the delay between shot and kill
- Noisy nets explore without a hand-tuned epsilon schedule

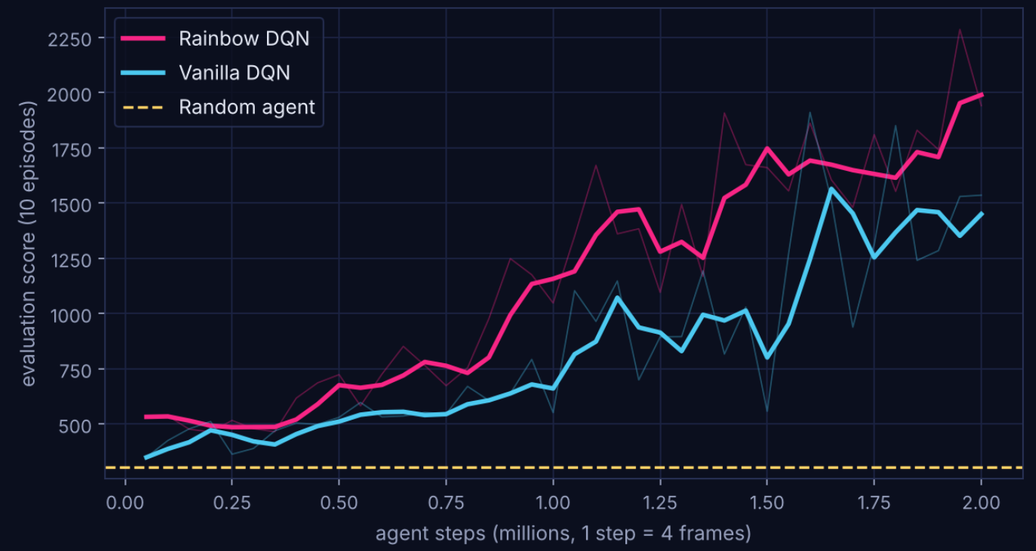
Expected challenges

- Compute: the paper used 200M frames; we can afford ~8M
- High variance between episodes (random fire)
- Delayed reward and clipped rewards (all kills = +1)
- Many hyper-parameters; sensitive to bugs
- Shaped rewards may be exploited (reward hacking)

Training and Evaluation Plan

- **Training** — 2M agent steps (8M frames), RTX 5070, about 2 h per run; Adam lr 1e-4, batch 32, replay 200k, target update 2k.
- **During training** — every 50k steps: 10 evaluation games, save the best model.
- **Final evaluation** — 30 games on the same seeds for every agent; mean / median score, waves cleared, shooting accuracy.
- **Baselines** — random agent and vanilla DQN with the same budget.
- **If time allows** — ablations (Rainbow minus one component), shaped reward, difficulty curriculum.

Pilot run: the pipeline already works



30 test games: Random 373 · DQN 1,462 · Rainbow 2,073 (5.6x random)

THANK YOU · QUESTIONS?

References

Mnih et al. (2015). Human-level control through deep reinforcement learning. Nature.

Hessel et al. (2018). Rainbow: Combining Improvements in Deep Reinforcement Learning. AAAI.

van Hasselt, Guez, Silver (2016). Deep RL with Double Q-learning. AAAI.

Schaul et al. (2016). Prioritized Experience Replay. ICLR.

Wang et al. (2016). Dueling Network Architectures for Deep RL. ICML.

Bellemare, Dabney, Munos (2017). A Distributional Perspective on RL. ICML.

Fortunato et al. (2018). Noisy Networks for Exploration. ICLR.

Obando-Ceron & Castro (2021). Revisiting Rainbow. ICML.